This LM is to add security group module and use it in the parent main.tf

Steps

1. Create the security group module

In the module github, create an **“security-groups”** folder

**Filename1: sg.tf**

```
# Security Group - base security group (rules defined separately)
resource "aws_security_group" "web_sg" {
  name        = "lab9-web-sg-${var.project}"
  description = "Security group for web server"
  vpc_id      = var.vpc_id

  tags = {
    Name = "lab9-web-sg-${var.project}"
  }
}

# Security Group Rule - HTTP ingress
resource "aws_vpc_security_group_ingress_rule" "http" {
  security_group_id = aws_security_group.web_sg.id
  description       = "HTTP"
  from_port         = 80
  to_port           = 80
  ip_protocol       = "tcp"
  cidr_ipv4         = "0.0.0.0/0"

  tags = {
    Name = "lab9-http-rule-${var.project}"
  }
}

# Security Group Rule - ssh ingress
```

```
resource "aws_vpc_security_group_ingress_rule" "ssh" {
  security_group_id = aws_security_group.web_sg.id
  description       = "ssh"
  from_port         = 22
  to_port           = 22
  ip_protocol       = "tcp"
  cidr_ipv4         = "0.0.0.0/0"

  tags = {
    Name = "lab9-http-rule-${var.project}"
  }
}

# Security Group Rule - Allow all outbound
resource "aws_vpc_security_group_egress_rule" "all_outbound" {
  security_group_id = aws_security_group.web_sg.id
  description       = "Allow all outbound"
  ip_protocol       = "-1"
  cidr_ipv4         = "0.0.0.0/0"

  tags = {
    Name = "lab9-egress-rule-${var.project}"
  }
}
```

Filename2: sg_var.tf

```
variable "vpc_id" {
  description = "VPC ID where security groups will be created"
  type        = string
}
```

*variable "project" {*

*description = "Project name - use UserX format where X is your student number"*

*type = string*

*}*

Filename3: sg_output.tf

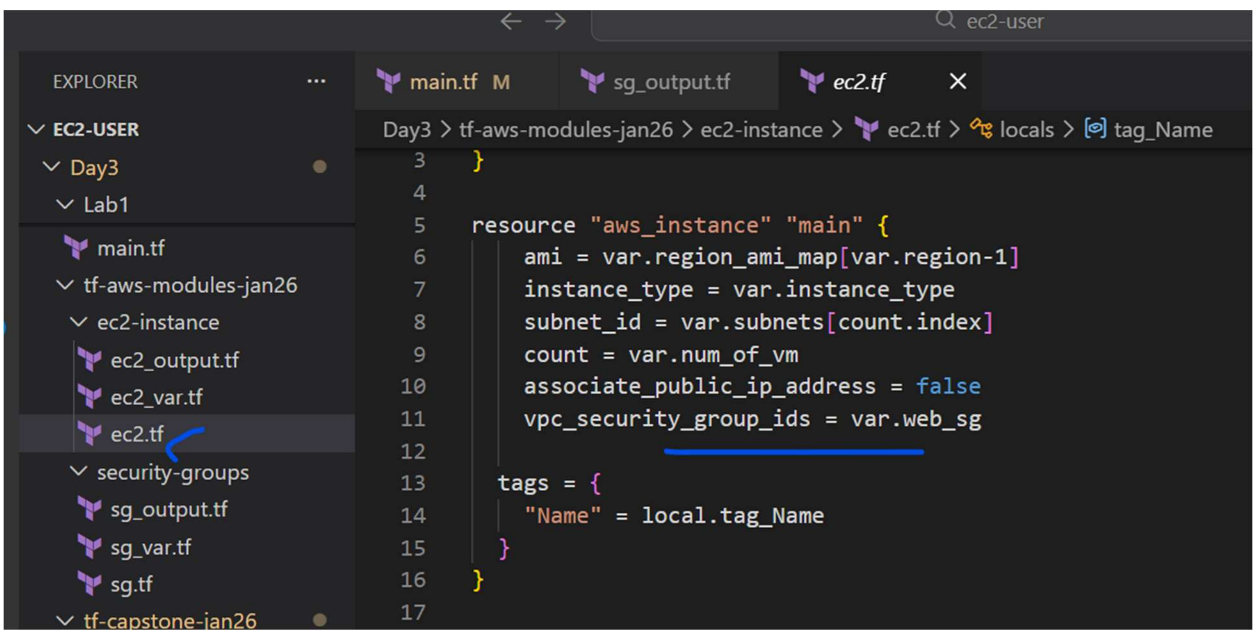
*output "web_sg_id" {*

*description = "ID of the web security group"*

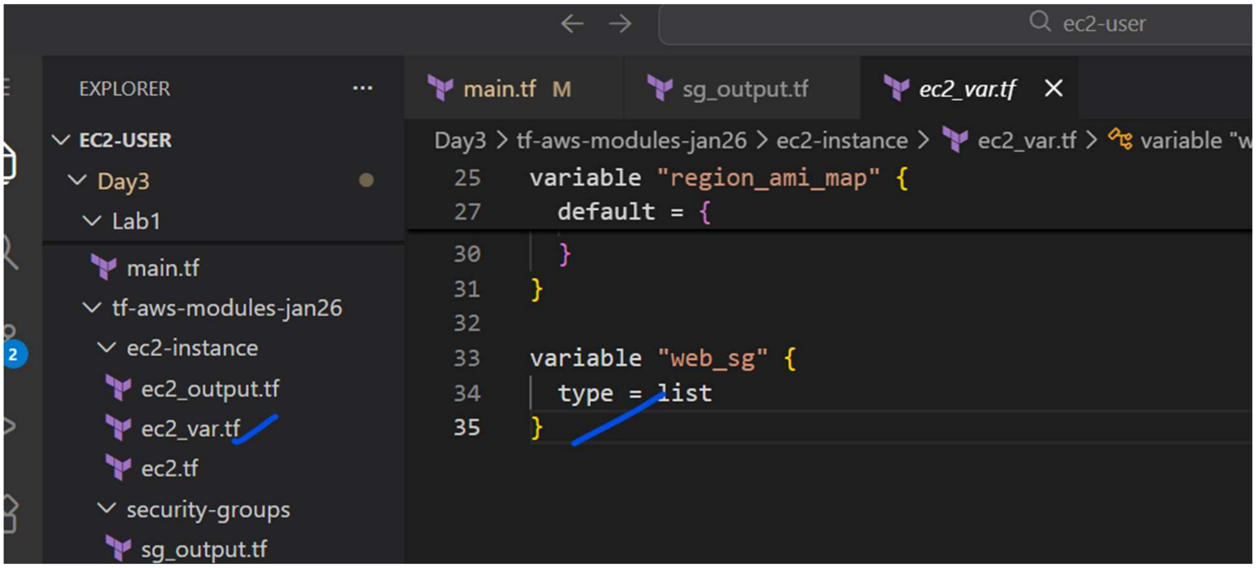
*value = aws_security_group.web_sg.id*

*}*

2. Add security group argument in the “ec2-instance” module

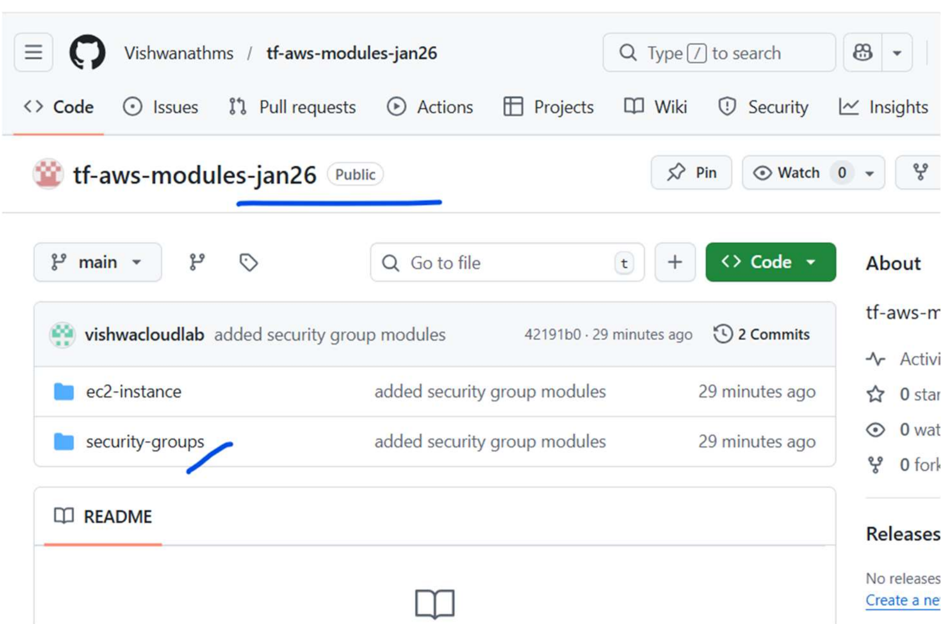



Add the below to the “ec2_var.tf file




*variable "web_sg" {*
*type = list*
*}*

3. Push the changes to the module github repo

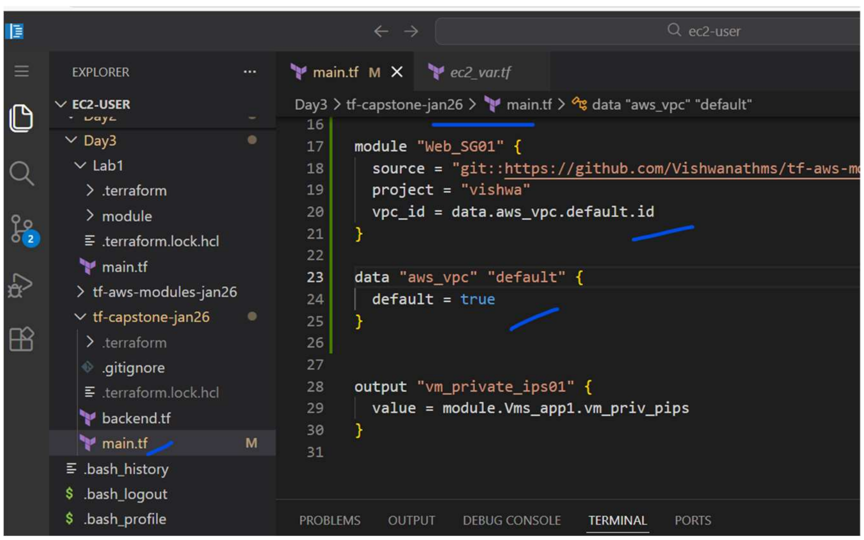


4. Add the module to the main.tf

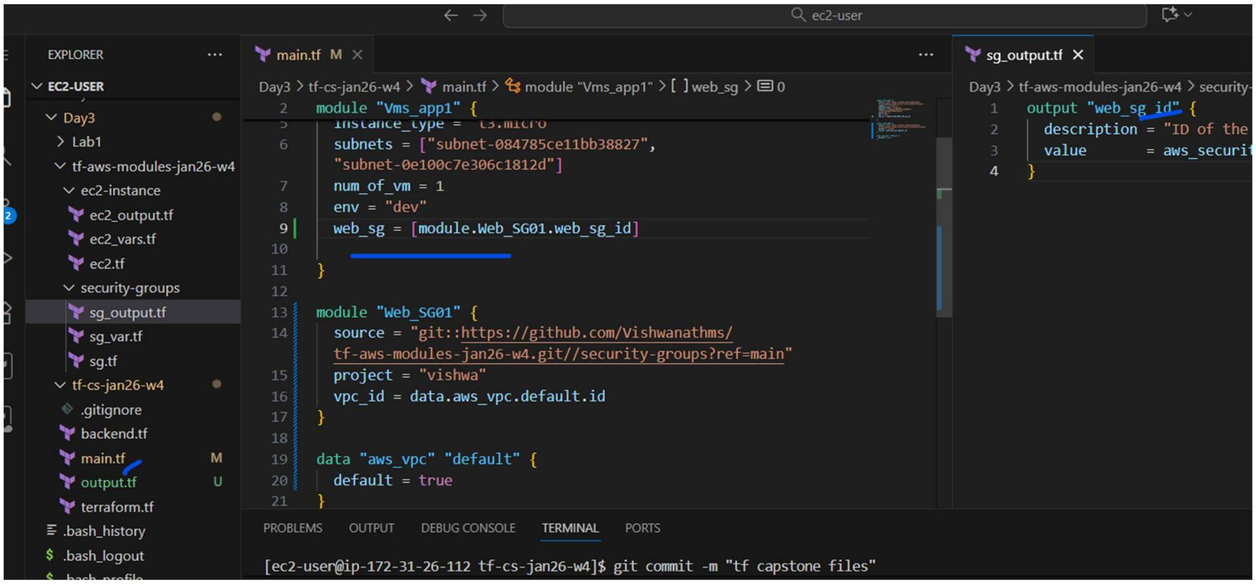
   We are using the data to fetch the VPC ID automatically, instead of hard coding.

```
module "Web_SG01" {
  source = "git::https://github.com/Vishwanathms/tf-aws-modules-
jan26.git//security-groups?ref=main"
  project = "vishwa"
  vpc_id = data.aws_vpc.default.id
}

data "aws_vpc" "default" {
  default = true
}
```

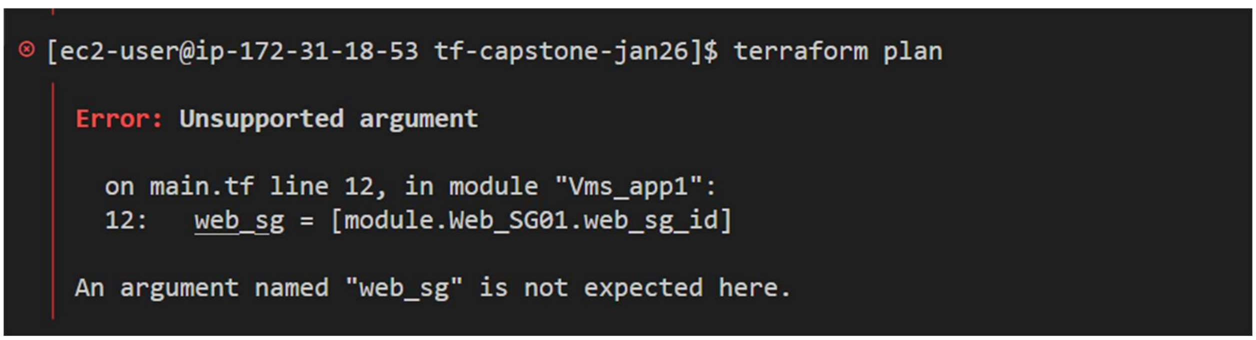



5. Update the “vms_app1” module with “web_sg” variable

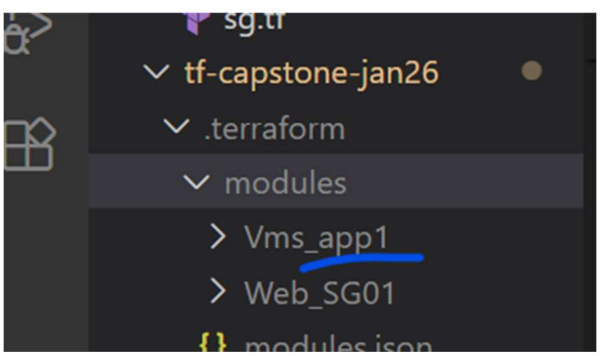



6. Run terraform init and plan

We will get the below error

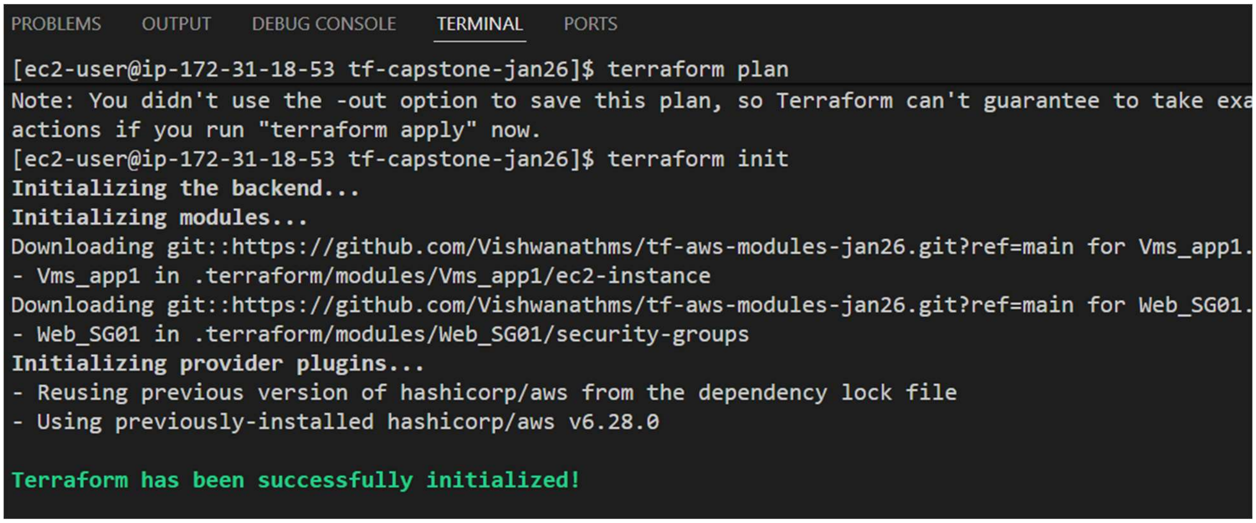



This is bcz the cached module folder in the .terraform in the capstone has the old module data for "Vms_app1"




Delete that folder "Vms_app1" and run

Terraform init




And then the terraform plan and apply will work

Output.